

1. Objetivo

Aprovechar el paralelismo de una GPU para optimizar algoritmos.

2. Ejercicio

Lanzar un *kernel* de un solo bloque que ordene de menor a mayor un vector de *N* elementos definido en tiempo de ejecución por el usuario con valores aleatorios comprendidos entre 1 y 30. La ordenación debe realizarse utilizando el método de ordenación por rango, donde cada hilo de ejecución del *kernel* debe colocar un elemento en el vector final a partir de su rango.

El programa debe mostrar las propiedades del dispositivo seleccionado (nombre, capacidad de cómputo, número de multiprocesadores, número de núcleos y máximo número de hilos por bloque), el número de elementos a ordenar y además:

» El vector de entrada desordenado.

» El vector de salida ordenado.

» El tiempo de ejecución del *kernel* en ms.

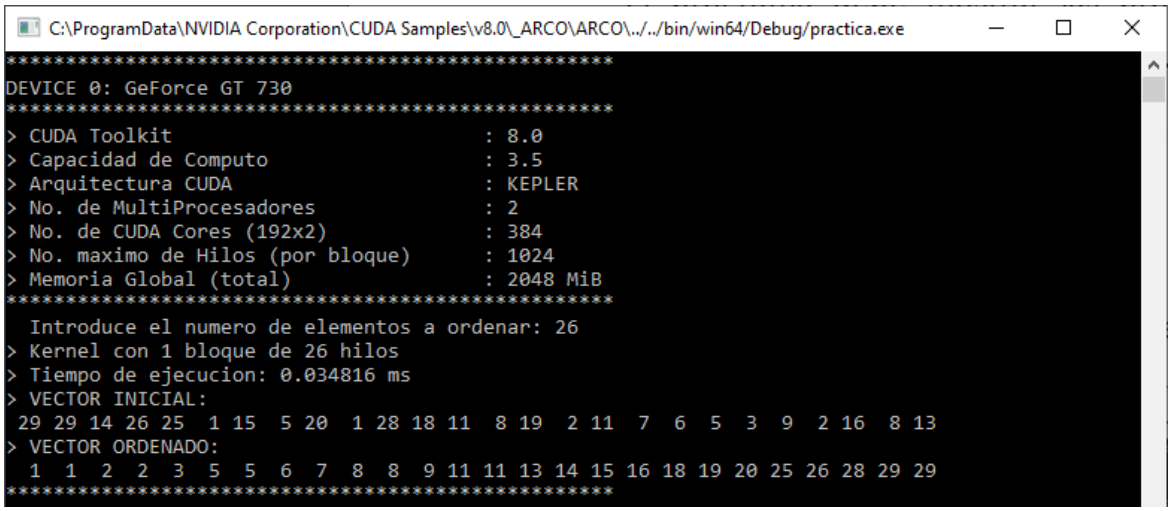


Figura 4.1.: Salida por pantalla de una posible solución.

3. Algoritmo de ordenación por rango

El algoritmo de ordenación por rango es muy sencillo aunque poco óptimo para ser implementado de forma secuencial. El algoritmo consiste en comparar cada elemento a ordenar con todos los demás con el fin de averiguar su rango. El rango de un elemento se refiere a su posición dentro el vector final. Si disponemos de n elementos, el algoritmo implementado de forma secuencial tiene una complejidad de orden $O(n^2)$, pero una implementación paralela da lugar una complejidad de orden O(n).

En un ordenamiento de menor a mayor, para calcular el rango de un elemento basta con contar el número de elementos que son más pequeños que él. En el caso de que dos elementos sean iguales, se considera que es más pequeño el que originalmente se encuentra en una posición más baja. Por ejemplo, dada la siguiente lista de elementos:

```
vector_inicio = [ 22 20 11 30 14 20 18 25 12 23 ]
```

donde hay dos elementos repetidos, el rango de cada uno de ellos sería, respectivamente:

```
( 6  4  0  9  2  5  3  8  1  7 )
```

Utilizando estos rangos podemos generar la lista de elementos ordenados de menor a mayor, la cual quedaría del siguiente modo:

```
vector_final =  [ 11 12 14 18 20 20 22 23 25 30 ]
```